

Smart Home Demo Lab Team 2 Update 4 Report

Yixue Wang, Linghao Dong, Yang Xiao, Chenxu Liu, Tong Lu

December 10, 2025

Overview of Work in the Past Two Weeks

Application Layer

- **DeviceService**

Device registration, enable/disable, state sync.

Overview of Work in the Past Two Weeks

Application Layer

- **DeviceService**
Device registration, enable/disable, state sync.
- **SceneService**
Scene creation, editing, publishing, disabling.

Overview of Work in the Past Two Weeks

Application Layer

- **DeviceService**
Device registration, enable/disable, state sync.
- **SceneService**
Scene creation, editing, publishing, disabling.
- **OrchestrationService**
Scene execution coordination & workflow management.

Overview of Work in the Past Two Weeks

Application Layer

- **DeviceService**
Device registration, enable/disable, state sync.
- **SceneService**
Scene creation, editing, publishing, disabling.
- **OrchestrationService**
Scene execution coordination & workflow management.

Overview of Work in the Past Two Weeks

Application Layer

- **DeviceService**
Device registration, enable/disable, state sync.
- **SceneService**
Scene creation, editing, publishing, disabling.
- **OrchestrationService**
Scene execution coordination & workflow management.

Infrastructure Layer

- **Data Persistence**
SQLAlchemy ORM, Repositories.

Overview of Work in the Past Two Weeks

Application Layer

- **DeviceService**
Device registration, enable/disable, state sync.
- **SceneService**
Scene creation, editing, publishing, disabling.
- **OrchestrationService**
Scene execution coordination & workflow management.

Infrastructure Layer

- **Data Persistence**
SQLAlchemy ORM, Repositories.
- **Event Bus**
asyncio-based in-memory event pub/sub.

Overview of Work in the Past Two Weeks

Application Layer

- **DeviceService**
Device registration, enable/disable, state sync.
- **SceneService**
Scene creation, editing, publishing, disabling.
- **OrchestrationService**
Scene execution coordination & workflow management.

Infrastructure Layer

- **Data Persistence**
SQLAlchemy ORM, Repositories.
- **Event Bus**
asyncio-based in-memory event pub/sub.
- **Hardware Communication**
HTTP Client & Client Registry.

Overview of Work in the Past Two Weeks

Application Layer

- **DeviceService**
Device registration, enable/disable, state sync.
- **SceneService**
Scene creation, editing, publishing, disabling.
- **OrchestrationService**
Scene execution coordination & workflow management.

Infrastructure Layer

- **Data Persistence**
SQLAlchemy ORM, Repositories.
- **Event Bus**
asyncio-based in-memory event pub/sub.
- **Hardware Communication**
HTTP Client & Client Registry.
- **Logging Module**
Unified logging configuration.



Click to Play Demo Video

Application Layer - Device Service

Unified management of smart device lifecycle, providing registration, state sync, and control capabilities.

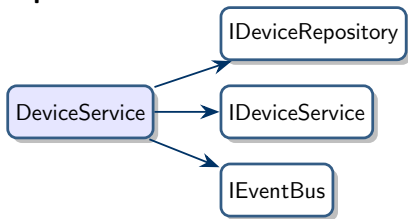
Design Concept: Acts as a mediator, encapsulating device management flows, coordinating domain objects, and publishing events.

Application Layer - Device Service

Unified management of smart device lifecycle, providing registration, state sync, and control capabilities.

Design Concept: Acts as a mediator, encapsulating device management flows, coordinating domain objects, and publishing events.

Dependencies:

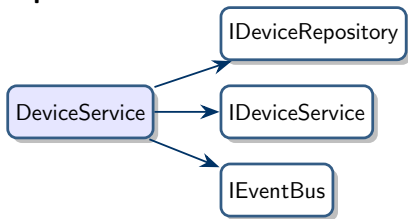


Application Layer - Device Service

Unified management of smart device lifecycle, providing registration, state sync, and control capabilities.

Design Concept: Acts as a mediator, encapsulating device management flows, coordinating domain objects, and publishing events.

Dependencies:



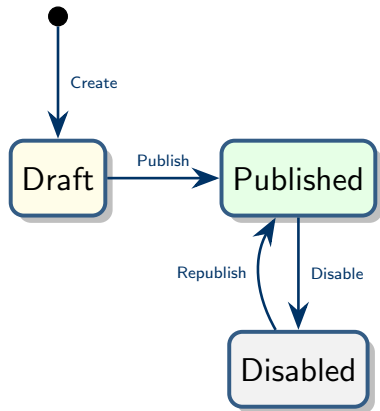
Functional List

Method	Description
register_device()	Register, generate UUID, publish event
enable/disable	Toggle state, publish status change
sync_device_state()	Delegate domain service to sync state
get_device(.by_id)	Get device details
list_devices()	Query list (with filters)
delete_device()	Delete device

Application Layer - Scene Service

Manages the full lifecycle of scenes and validates scene definitions.

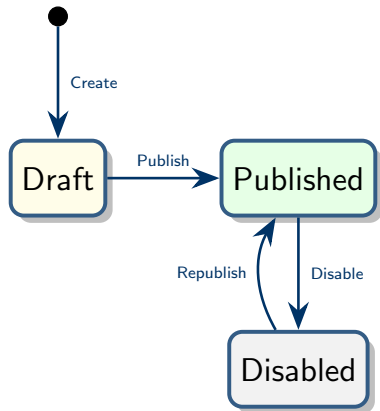
Scene State Machine



Application Layer - Scene Service

Manages the full lifecycle of scenes and validates scene definitions.

Scene State Machine



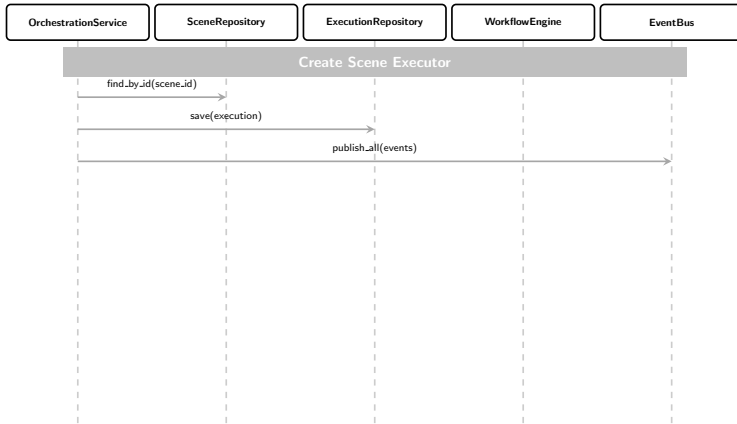
Functional List

Method	Description
<code>create_scene()</code>	Create draft scene
<code>update_scene(_def)</code>	Update info/def (w/ validation)
<code>publish_scene()</code>	Publish (triggers event)
<code>disable_scene()</code>	Disable (triggers event)
<code>get_scene(_def)</code>	Get details/definition
<code>list_scenes</code>	Query list
<code>delete_scene()</code>	Delete scene

Application Layer - Orchestration Service

Acts as the coordinator of three contexts, triggers execution, and drives the workflow.

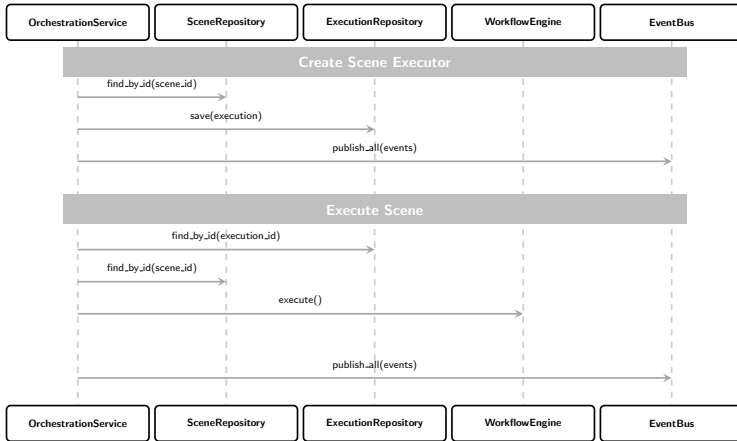
Execution Flow



Application Layer - Orchestration Service

Acts as the coordinator of three contexts, triggers execution, and drives the workflow.

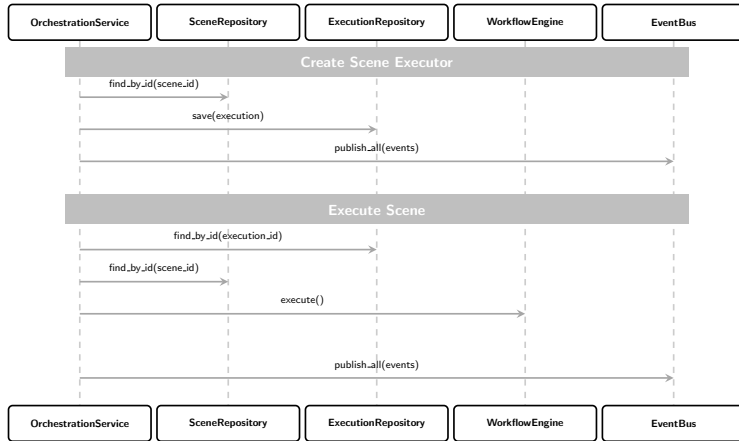
Execution Flow



Application Layer - Orchestration Service

Acts as the coordinator of three contexts, triggers execution, and drives the workflow.

Execution Flow



Functional List

Method	Description
<code>trigger_execution()</code>	Triggers execution, creates record
<code>execute_scene()</code>	Invokes engine to execute scene
<code>trigger_and_execute()</code>	Trigger & execute immediately
<code>get_execution()</code>	Retrieves execution record
<code>get_execution_details()</code>	Gets structured exec details
<code>list_executions()</code>	Queries execution list
<code>list_executions_for_scene()</code>	Gets all executions for a scene

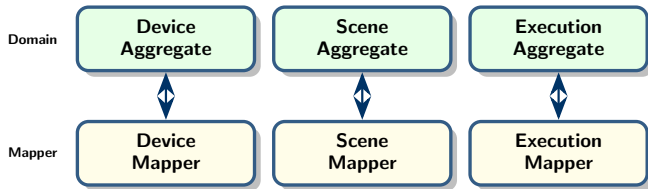
Infrastructure Layer - Data Persistence

Architecture: Implementation based on SQLAlchemy, supporting SQLite/PostgreSQL.



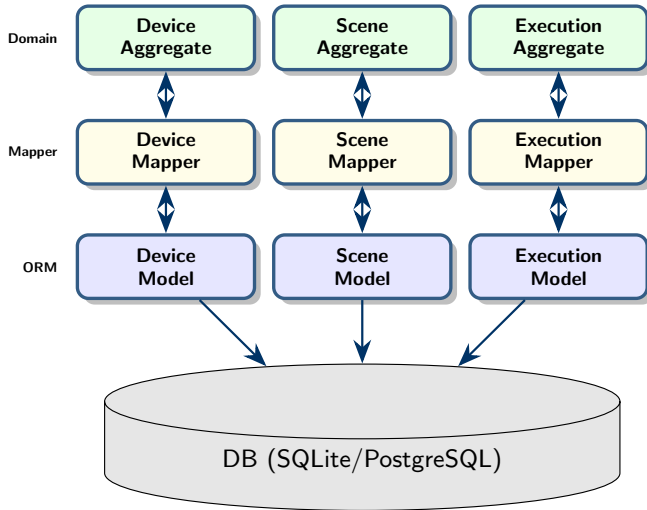
Infrastructure Layer - Data Persistence

Architecture: Implementation based on SQLAlchemy, supporting SQLite/PostgreSQL.



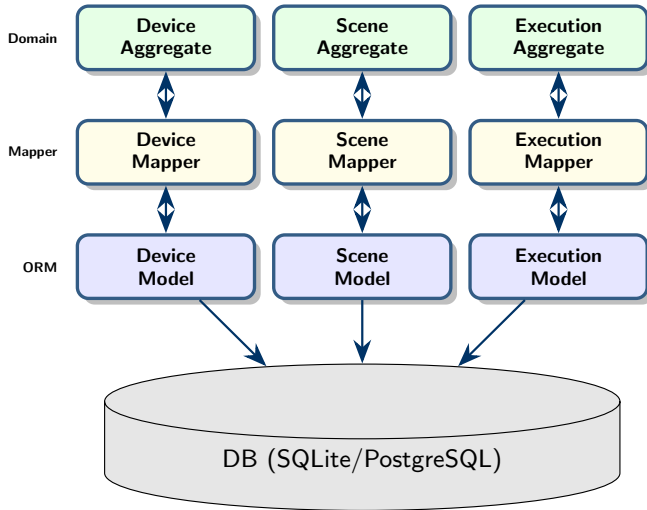
Infrastructure Layer - Data Persistence

Architecture: Implementation based on SQLAlchemy, supporting SQLite/PostgreSQL.



Infrastructure Layer - Data Persistence

Architecture: Implementation based on SQLAlchemy, supporting SQLite/PostgreSQL.

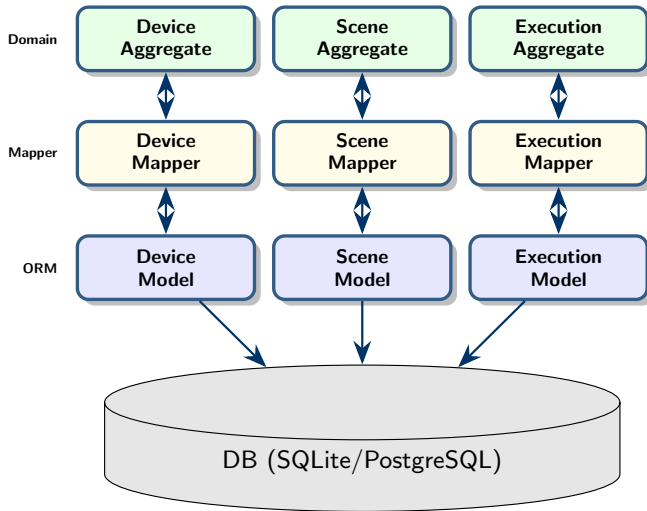


Key Features

- **Isolation:** Strict boundaries.

Infrastructure Layer - Data Persistence

Architecture: Implementation based on SQLAlchemy, supporting SQLite/PostgreSQL.

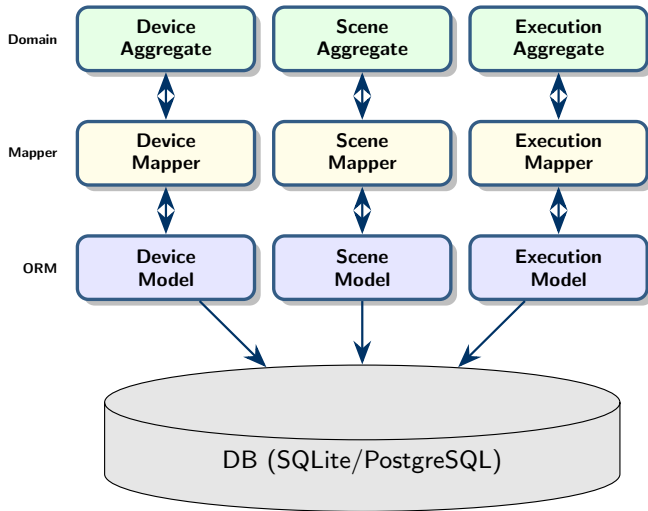


Key Features

- **Isolation:** Strict boundaries.
- **Mapping:** Domain $\leftrightarrow$ ORM.

Infrastructure Layer - Data Persistence

Architecture: Implementation based on SQLAlchemy, supporting SQLite/PostgreSQL.



Key Features

- **Isolation:** Strict boundaries.
- **Mapping:** Domain $\leftrightarrow$ ORM.
- **Repository:** Encapsulates data access.

Infrastructure Layer - Event Bus

Decouples context communication, based on `asyncio.PriorityQueue`.

Features

- **Priority:** HIGH > NORMAL > LOW

Infrastructure Layer - Event Bus

Decouples context communication, based on `asyncio.PriorityQueue`.

Features

- **Priority:** HIGH > NORMAL > LOW
- **Lifecycle:** `start()` / `stop()`

Infrastructure Layer - Event Bus

Decouples context communication, based on `asyncio.PriorityQueue`.

Features

- **Priority:** HIGH > NORMAL > LOW
- **Lifecycle:** `start()` / `stop()`
- **Handlers:** Sync/Async support

Infrastructure Layer - Event Bus

Decouples context communication, based on `asyncio.PriorityQueue`.

Features

- **Priority:** HIGH > NORMAL > LOW
- **Lifecycle:** `start()` / `stop()`
- **Handlers:** Sync/Async support
- **Extensibility:** Interface ready for MQ

Infrastructure Layer - Event Bus

Decouples context communication, based on `asyncio.PriorityQueue`.

Features

- **Priority:** HIGH > NORMAL > LOW
- **Lifecycle:** `start()` / `stop()`
- **Handlers:** Sync/Async support
- **Extensibility:** Interface ready for MQ

Infrastructure Layer - Event Bus

Decouples context communication, based on `asyncio.PriorityQueue`.

Features

- **Priority:** HIGH > NORMAL > LOW
- **Lifecycle:** `start()` / `stop()`
- **Handlers:** Sync/Async support
- **Extensibility:** Interface ready for MQ

Core Interface Definition

```
class IEventBus:
    # Publish single event
    async def publish(event, priority)

    # Publish batch
    async def publish_all(events)

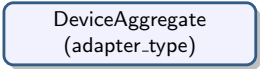
    # Subscribe to event type
    def subscribe(event_type, handler)

    # Unsubscribe
    def unsubscribe(event_type, handler)
```

Infrastructure Layer - Hardware Communication

Abstracts platform differences, provides unified control interface.

Layered Design Architecture

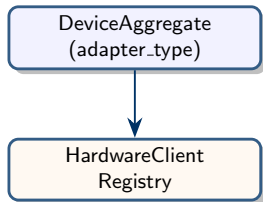


DeviceAggregate
(adapter_type)

Infrastructure Layer - Hardware Communication

Abstracts platform differences, provides unified control interface.

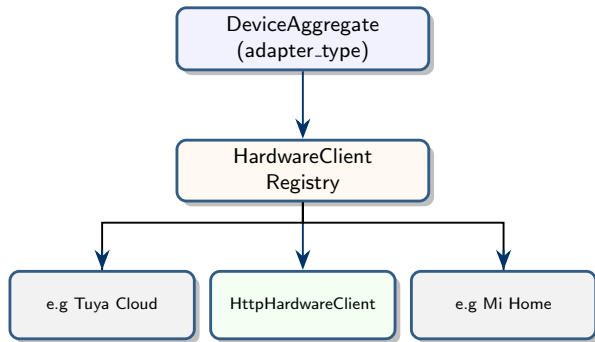
Layered Design Architecture



Infrastructure Layer - Hardware Communication

Abstracts platform differences, provides unified control interface.

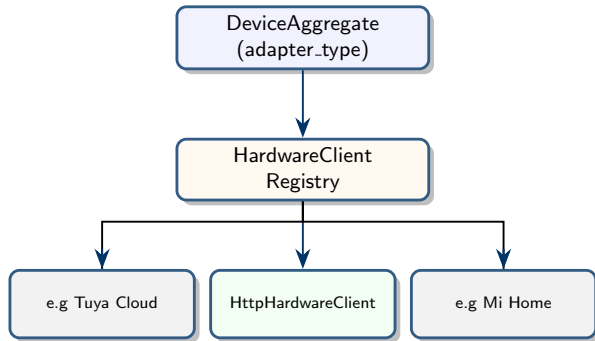
Layered Design Architecture



Infrastructure Layer - Hardware Communication

Abstracts platform differences, provides unified control interface.

Layered Design Architecture



HttpHardwareClient

Feature	Description
Platform	API
Auth	Token
Methods	call, get

Next Steps

Pending Tasks

#	Task	Description
1	Controllers Layer	REST API, DTO, FastAPI Config
2	Frontend	UI Development
3	E2E Testing	Full business flow verification
4	Documentation	API Docs, Architecture, Guides

Next Steps

Pending Tasks

#	Task	Description
1	Controllers Layer	REST API, DTO, FastAPI Config
2	Frontend	UI Development
3	E2E Testing	Full business flow verification
4	Documentation	API Docs, Architecture, Guides

Future Capabilities

- Sub-scene calls (Nested scenes)

Next Steps

Pending Tasks

#	Task	Description
1	Controllers Layer	REST API, DTO, FastAPI Config
2	Frontend	UI Development
3	E2E Testing	Full business flow verification
4	Documentation	API Docs, Architecture, Guides

Future Capabilities

- Sub-scene calls (Nested scenes)
- Scene version control

Next Steps

Pending Tasks

#	Task	Description
1	Controllers Layer	REST API, DTO, FastAPI Config
2	Frontend	UI Development
3	E2E Testing	Full business flow verification
4	Documentation	API Docs, Architecture, Guides

Future Capabilities

- Sub-scene calls (Nested scenes)
- Scene version control
- Containerization support

Next Steps

Pending Tasks

#	Task	Description
1	Controllers Layer	REST API, DTO, FastAPI Config
2	Frontend	UI Development
3	E2E Testing	Full business flow verification
4	Documentation	API Docs, Architecture, Guides

Future Capabilities

- Sub-scene calls (Nested scenes)
- Scene version control
- Containerization support

Next Steps

Pending Tasks

#	Task	Description
1	Controllers Layer	REST API, DTO, FastAPI Config
2	Frontend	UI Development
3	E2E Testing	Full business flow verification
4	Documentation	API Docs, Architecture, Guides

Future Capabilities

- Sub-scene calls (Nested scenes)
- Scene version control
- Containerization support

Team Members: Yixue Wang, Linghao Dong, Yang Xiao, Chenxu Liu, Tong Lu

Report Date: December 10, 2025